
Local Density Estimation and Clustering

Local density estimation and mean-shift clustering

Kangning Cui

City University of Hong Kong (Dongguan)

Semester A, 2026/27

Learning objectives

Replace fixed mixture shapes with density determined by local data

1. explain why a fixed distribution family may be too restrictive;
2. estimate a density by adding local contributions from data points;
3. explain how neighborhood width changes smoothness and variability;
4. derive the mean-shift update from the estimated density;
5. compare K-means, Gaussian mixtures, and mean shift.

Local density estimates

CS5487 ■ Semester A, 2026/27

Global density models

Suppose we observe n data points:

$$x_1, \dots, x_n.$$

A parametric model describes the whole dataset using a fixed distribution family.
For example:

$$\begin{aligned}\text{Gaussian} &\rightarrow (\mu, \sigma^2), \\ \text{GMM} &\rightarrow \{(\pi_k, \mu_k, \sigma_k^2)\}_{k=1}^K.\end{aligned}$$

More data improve the parameter estimates, but the basic model form remains fixed.

Local methods instead let the observed neighborhoods determine the density shape.

Local data concentration

Consider a location x where we want to estimate the density.

Choose a neighborhood radius:

$$r > 0.$$

An observation x_i is considered nearby when:

$$|x_i - x| \leq r,$$

where $|x_i - x|$ is the distance between x_i and x .

The local neighborhood is therefore:

$$\mathcal{N}_r(x) = \{x_i : |x_i - x| \leq r\}.$$

Let:

$$N_r(x) = |\mathcal{N}_r(x)|$$

be the number of observations inside this neighborhood.

For the same radius r , a larger $N_r(x)$ indicates greater local concentration.

Local probability mass

In one dimension, the neighborhood around x is:

$$[x - r, x + r].$$

Its total width is:

$$(x + r) - (x - r) = 2r.$$

The probability mass inside this interval is:

$$P(x - r \leq X \leq x + r) = \int_{x-r}^{x+r} p(u) du.$$

If the interval is small enough that $p(u)$ changes little around x :

$$p(u) \approx p(x).$$

Therefore:

$$P(x - r \leq X \leq x + r) \approx 2r p(x).$$

From local counts to density

The same local probability can be estimated from the observed sample. Among n observations, $N_r(x)$ fall inside the interval:

$$[x - r, x + r].$$

Hence the empirical probability mass is:

$$P(x - r \leq X \leq x + r) \approx \frac{N_r(x)}{n}.$$

Combining this with:

$$P(x - r \leq X \leq x + r) \approx 2r p(x),$$

gives:

$$\hat{p}_r(x) = \frac{N_r(x)}{2nr}.$$

So:

$$\text{density} = \frac{\text{local sample fraction}}{\text{local width}}.$$

Histogram density estimation

A histogram applies the same idea using fixed bins.

Let:

$$h > 0$$

be the bin width, and let $N_b(x)$ be the number of observations in the bin containing x .

The empirical probability mass of that bin is:

$$\frac{N_b(x)}{n}.$$

Dividing by the bin width gives the density:

$$\hat{p}(x) = \frac{N_b(x)}{nh}.$$

The estimate is constant throughout each bin.

Bin width controls resolution

The bin width h determines the amount of local averaging.

For small h :

fewer points per bin $\rightarrow$ more detail and more variability.

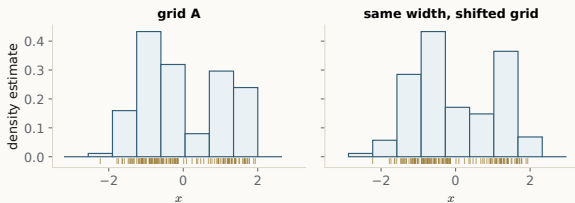
For large h :

more points per bin $\rightarrow$ more smoothing and less detail.

Thus h controls the resolution of the density estimate.

The same role will later be played by the bandwidth in KDE.

Histogram boundary effects



The bin grid is chosen arbitrarily.
Shifting the boundaries can change:

$$N_b(x)$$

and therefore the estimated density.
Two limitations follow:

- ▶ discontinuous jumps between bins;
- ▶ sensitivity to grid placement.

KDE removes fixed bin boundaries
by centering a smooth
neighborhood on each observation.

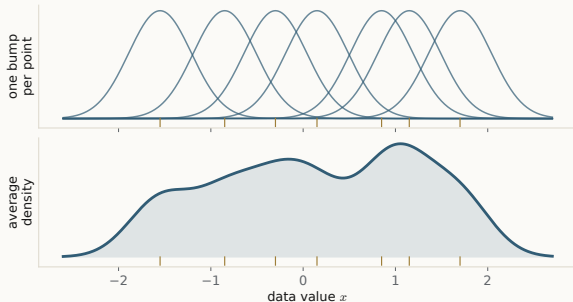
Kernel density estimation

CS5487 ■ Semester A, 2026/27

Smooth contributions around observations

A histogram groups observations into fixed bins.

Kernel density estimation (KDE) places a local contribution around each observation and averages them.



With a smooth kernel, nearby observations create overlapping contributions and a smooth density curve.

Kernel and bandwidth

Each observation x_i contributes a function:

$$q_{i,h}(x) = \frac{1}{h} K \left(\frac{x - x_i}{h} \right), \quad h > 0.$$

- ▶ The **kernel** K determines the shape.
- ▶ The **bandwidth** h determines the width.
- ▶ The factor $1/h$ adjusts the height to preserve area.

The quantity $(x - x_i)/h$ measures distance from x_i in bandwidth units.
For a Gaussian kernel, increasing h makes the contribution wider and lower.

From histograms to KDE

Histograms estimate density using fixed bins.

The limitations are:

- ▶ the result depends on bin locations;
- ▶ the density changes abruptly at bin boundaries;
- ▶ nearby observations are treated identically inside one bin.

KDE keeps the idea of local counting, but changes the neighborhood:

fixed bins → smooth neighborhoods around observations.

KDE replaces hard bins with smooth contributions from individual data points.

One observation creates one density contribution

Suppose we observe one data point:

$$x_i.$$

Instead of putting all its probability mass into one bin, KDE spreads it smoothly around x_i .
The contribution is:

$$q_{i,h}(x) = \frac{1}{h} K \left(\frac{x - x_i}{h} \right).$$

Here:

- ▶ x_i : observed point;
- ▶ x : location where density is evaluated;
- ▶ h : neighborhood width;
- ▶ K : shape of the local contribution.

Each observation creates one local density bump.

Combining all local contributions

A dataset contains:

$$x_1, \dots, x_n.$$

KDE averages the contribution from every observation:

$$\hat{p}_h(x) = \frac{1}{n} \sum_{i=1}^n q_{i,h}(x)$$

or equivalently:

$$\hat{p}_h(x) = \frac{1}{nh} \sum_{i=1}^n K\left(\frac{x - x_i}{h}\right)$$

The density at x is determined by all nearby observations.

The role of the kernel

The kernel determines how one observation influences nearby locations.

A valid kernel satisfies:

$$K(u) \geq 0,$$

and:

$$\int K(u) du = 1.$$

Therefore one kernel represents a probability distribution with total mass one.

Common choices:

Gaussian kernel smooth contribution,

Box kernel constant contribution inside a window.

The kernel controls the shape, but usually has less influence than bandwidth.

Why does KDE contain the factor $1/h$?

The bandwidth changes the width of each contribution.

Larger h :

wider bump.

Smaller h :

narrower bump.

Changing the width should not change the total probability mass. For one observation:

$$q_{i,h}(x) = \frac{1}{h} K\left(\frac{x - x_i}{h}\right).$$

Using:

$$u = \frac{x - x_i}{h}, \quad dx = h \, du,$$

we obtain:

$$\int q_{i,h}(x) dx = \int K(u) du = 1.$$

Why is KDE a valid density?

A density must satisfy two properties:

$$\hat{p}_h(x) \geq 0,$$

and:

$$\int \hat{p}_h(x) dx = 1.$$

Because KDE is an average of normalized kernels:

$$\begin{aligned} \int \hat{p}_h(x) dx &= \frac{1}{n} \sum_i \int q_{i,h}(x) dx \\ &= \frac{1}{n} \sum_i 1 = 1. \end{aligned}$$

Therefore KDE is a valid probability density estimate.

Density is not probability

The KDE value:

$$\hat{p}_h(x)$$

is a density height, not a probability.

A probability requires an interval:

$$P(a \leq X \leq b) = \int_a^b \hat{p}_h(x) dx.$$

For a small interval:

$$P(x \leq X \leq x + \Delta) \approx \hat{p}_h(x) \Delta.$$

Therefore a density value can be larger than one.

For example, a narrow Gaussian bump may have a high peak but still have total area one.

Gaussian kernel

The standard Gaussian kernel is:

$$K(u) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{u^2}{2}\right).$$

Observation x_i contributes:

$$q_{i,h}(x) = \frac{1}{h\sqrt{2\pi}} \exp\left(-\frac{(x - x_i)^2}{2h^2}\right).$$

This is a Gaussian density with mean x_i and standard deviation h .

Nearby observations contribute more to the estimate at x . Distant observations contribute less, but their contributions remain positive.

A box-kernel example

The box kernel is:

$$K(u) = \frac{1}{2}\mathbb{I}(|u| \leq 1).$$

Its KDE has a simple counting interpretation:

$$\hat{p}_h(x) = \frac{\#\{i : |x - x_i| \leq h\}}{2nh}.$$

The numerator counts observations inside $[x - h, x + h]$. The denominator is sample size times window width.

Consider:

$$x_1 = 0, \quad x_2 = 1, \quad h = 1.$$

The two contributions overlap over the interval $[0, 1]$.

Local density calculation

For $x_1 = 0$, $x_2 = 1$ and $h = 1$:

$$\hat{p}_1(x) = \frac{\#\{i : |x - x_i| \leq 1\}}{4}.$$

Location x	Observations in window	$\hat{p}_1(x)$
-0.5	1	1/4
0.5	2	1/2
1.5	1	1/4
3	0	0

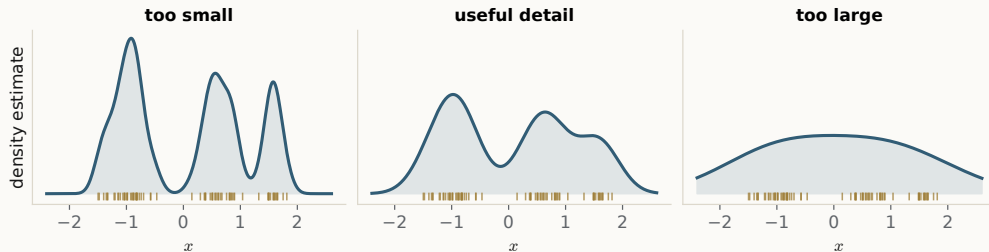
At $x = 0.5$, both observations contribute:

$$\hat{p}_1(0.5) = \frac{1}{2} \left(\frac{1}{2} + \frac{1}{2} \right) = \frac{1}{2}.$$

The estimate is higher where more local contributions overlap.

Density estimates across bandwidths

The same observations can produce very different estimates.



- ▶ **Small h :** narrow peaks and greater sensitivity to individual observations.
- ▶ **Moderate h :** a balance between detail and smoothing.
- ▶ **Large h :** broad contributions that may merge distinct features.

Multidimensional KDE

For observations $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, a scalar-bandwidth KDE is:

$$\hat{p}_h(\mathbf{x}) = \frac{1}{nh^d} \sum_{i=1}^n K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right).$$

Here K is a d -dimensional density:

$$K(\mathbf{u}) \geq 0, \quad \int_{\mathbb{R}^d} K(\mathbf{u}) d\mathbf{u} = 1.$$

Scaling every coordinate by h multiplies volume by h^d .

The factor $1/h^d$ preserves the area or volume under each contribution.

Gaussian kernel in multiple dimensions

The isotropic Gaussian kernel is:

$$K(\mathbf{u}) = (2\pi)^{-d/2} \exp\left(-\frac{1}{2}\|\mathbf{u}\|_2^2\right).$$

Observation $\mathbf{x}_i$ contributes:

$$q_{i,h}(\mathbf{x}) = \frac{1}{(2\pi)^{d/2}h^d} \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_i\|_2^2}{2h^2}\right).$$

Each contribution has mean $\mathbf{x}_i$ and covariance $h^2 I_d$.

A shared bandwidth uses the same distance scale in every coordinate. Feature scales therefore matter.

KDE as a smoothed density

KDE does not directly recover the true density.
Instead, each observation contributes a smooth local estimate:

$$x_i \rightarrow K_h(x - x_i).$$

Averaging over possible datasets gives:

$$\mathbb{E}[\hat{p}_h(x)] = \int K_h(x - u)p(u) du.$$

This is a convolution:

$$\boxed{\mathbb{E}[\hat{p}_h(x)] = (K_h * p)(x)}$$

Therefore KDE estimates a smoothed version of the true density.

The effect of smoothing

Because KDE averages nearby observations, sharp structures may be changed.
With larger bandwidth:

$$h \uparrow$$

the estimate becomes smoother:

peaks become lower,

valleys become filled.

With smaller bandwidth:

$$h \downarrow$$

more local details are preserved, but the estimate becomes more sensitive to individual samples.

Bias and variance trade-off

KDE error comes from two sources:

smoothing error, and sampling noise.

Small bandwidth:

$h \downarrow$

- ▶ lower smoothing bias;
- ▶ higher variance.

Large bandwidth:

$h \uparrow$

- ▶ higher smoothing bias;
- ▶ lower variance.

A good bandwidth balances these two effects.

The role of sample size

More observations reduce random fluctuations:

$n \uparrow \Rightarrow$ more reliable local estimates.

However, increasing the number of samples does not remove smoothing:

fixed $h \Rightarrow$ same level of smoothing.

Therefore KDE requires choosing both:

n and h

appropriately.

Local estimation in high dimensions

In d dimensions, a neighborhood with width h has volume:

$$h^d.$$

The effective local information depends on:

$$nh^d.$$

As dimension increases:

$$h^d$$

shrinks rapidly.

Therefore a local region may contain very few observations.

Dimension	Local volume	Difficulty
Low	larger	easier
High	smaller	harder

Selecting the bandwidth

The bandwidth is a model choice.

A practical procedure:

1. choose several candidate bandwidths;
2. fit KDE using training data;
3. evaluate density on validation data;
4. select the best bandwidth.

Using validation observations:

$$L_{\text{val}}(h) = \frac{1}{m} \sum_j \log \hat{p}_h(x_j^{\text{val}}).$$

Choose:

$$h^* = \arg \max_h L_{\text{val}}(h)$$

Why not use training likelihood?

A training point always contributes its own kernel:

$$\frac{K(0)}{nh^d}.$$

When:

$$h \rightarrow 0,$$

this contribution becomes extremely large.

The model can obtain high training likelihood by creating very narrow spikes around observed points.

Therefore:

training fit $\neq$ good density estimate.

Validation avoids this problem by evaluating unseen observations.

Practical KDE considerations

Important choices in practice:

- ▶ **Feature scaling**

Distances determine neighborhoods, so feature scales matter.

- ▶ **Boundary effects**

Gaussian kernels may assign density outside the valid range.

- ▶ **Numerical stability**

Very small densities require careful computation of logarithms.

Kernel density estimation: summary

KDE estimates density by averaging local contributions:

$$\hat{p}_h(\mathbf{x}) = \frac{1}{nh^d} \sum_i K\left(\frac{\mathbf{x} - \mathbf{x}_i}{h}\right).$$

The main ideas:

- ▶ each observation contributes a smooth local density;
- ▶ bandwidth controls the resolution;
- ▶ more dimensions require more data;
- ▶ bandwidth selection is essential.

KDE replaces a fixed global density shape with a data-driven local estimate.

Mean-shift clustering

CS5487 ■ Semester A, 2026/27

Density modes as cluster centers

A **mode** is a local maximum of the estimated density:

$$\nabla \hat{p}(\boldsymbol{x}) = \mathbf{0}.$$

Mean shift defines clusters through these density peaks.

The basic idea is:

starting point $\rightarrow$ higher-density region $\rightarrow$ local mode.

Points that converge to the same mode are assigned to the same cluster.

Unlike K-means, the cluster centers are not initialized directly.

Mean shift starts from KDE

For a Gaussian kernel, ignore constants that do not depend on $\mathbf{x}$:

$$\hat{p}(\mathbf{x}) \propto \sum_{i=1}^n \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_i\|_2^2}{2h^2}\right).$$

Define:

$$w_i(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_i\|_2^2}{2h^2}\right).$$

Then:

$$\hat{p}(\mathbf{x}) \propto \sum_i w_i(\mathbf{x}).$$

Nearby observations receive larger weights.

Gradient of the Gaussian KDE

For one Gaussian weight:

$$w_i(\mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_i\|^2}{2h^2}\right).$$

Its gradient is:

$$\nabla w_i(\mathbf{x}) = \frac{1}{h^2} w_i(\mathbf{x}) (\mathbf{x}_i - \mathbf{x}).$$

Therefore:

$$\nabla \hat{p}(\mathbf{x}) \propto \frac{1}{h^2} \sum_i w_i(\mathbf{x}) (\mathbf{x}_i - \mathbf{x}).$$

The density gradient points toward observations with large local weights.

Weighted displacement

The weighted displacement can be rearranged:

$$\begin{aligned} & \sum_i w_i(\mathbf{x})(\mathbf{x}_i - \mathbf{x}) \\ &= \left(\sum_i w_i(\mathbf{x}) \right) \left[\frac{\sum_i w_i(\mathbf{x}) \mathbf{x}_i}{\sum_i w_i(\mathbf{x})} - \mathbf{x} \right]. \end{aligned}$$

Define the local weighted mean:

$$\mathbf{m}(\mathbf{x}) = \frac{\sum_i w_i(\mathbf{x}) \mathbf{x}_i}{\sum_i w_i(\mathbf{x})}.$$

The gradient therefore points from $\mathbf{x}$ toward $\mathbf{m}(\mathbf{x})$.

From KDE gradient to the mean-shift vector

For Gaussian KDE, we derived:

$$\nabla \hat{p}(\mathbf{x}) = \frac{1}{h^2} \sum_i w_i(\mathbf{x})(\mathbf{x}_i - \mathbf{x}).$$

Factor out the total weight:

$$\nabla \hat{p}(\mathbf{x}) = \frac{1}{h^2} \left(\sum_i w_i(\mathbf{x}) \right) \left[\frac{\sum_i w_i(\mathbf{x}) \mathbf{x}_i}{\sum_i w_i(\mathbf{x})} - \mathbf{x} \right].$$

The term inside the brackets is the displacement from the current point to the local weighted mean:

$$\boxed{\mathbf{s}(\mathbf{x}) = \mathbf{m}(\mathbf{x}) - \mathbf{x}}$$

The mean-shift update

Starting from:

$$\mathbf{x}^{(0)},$$

repeat:

$$w_i^{(t)} = \exp \left(-\frac{\|\mathbf{x}^{(t)} - \mathbf{x}_i\|^2}{2h^2} \right),$$

then compute:

$$\mathbf{x}^{(t+1)} = \frac{\sum_i w_i^{(t)} \mathbf{x}_i}{\sum_i w_i^{(t)}}.$$

The weights are recomputed after every update.

The iteration stops when:

$$\|\mathbf{x}^{(t+1)} - \mathbf{x}^{(t)}\|$$

becomes sufficiently small.

One mean-shift step

Consider one-dimensional observations:

$$x_1 = 0, \quad x_2 = 2, \quad x_3 = 4.$$

Start from:

$$x^{(0)} = 1, \quad h = 2.$$

The Gaussian weights are:

$$w_1 = e^{-1/8} \approx 0.8825,$$

$$w_2 = e^{-1/8} \approx 0.8825,$$

$$w_3 = e^{-9/8} \approx 0.3247.$$

Nearby observations contribute more strongly to the update.

Updating the location

Using the previous weights:

$$x^{(1)} = \frac{0.8825(0) + 0.8825(2) + 0.3247(4)}{0.8825 + 0.8825 + 0.3247}.$$

Therefore:

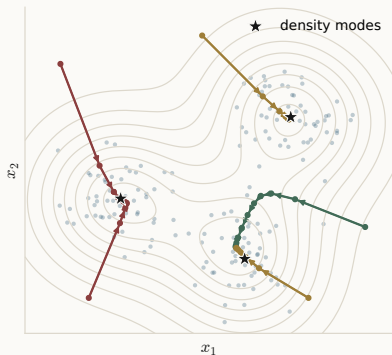
$$x^{(1)} \approx 1.47.$$

The point moves:

$$1 \rightarrow 1.47.$$

At the new location, the weights are recomputed and the process repeats.

Mean-shift trajectories



Different starting points may converge to the same mode:

$$\mathbf{x}_a^{(0)} \rightarrow \mathbf{m}_1,$$

$$\mathbf{x}_b^{(0)} \rightarrow \mathbf{m}_1.$$

They are assigned to the same cluster.
A mode therefore defines a local basin of attraction.

Bandwidth controls the discovered modes

Mean shift does not require a fixed number of clusters K .

Instead, the bandwidth controls the density landscape.

For small h :

many narrow density peaks $\rightarrow$ more clusters.

For large h :

nearby peaks merge $\rightarrow$ fewer clusters.

Thus:

bandwidth controls both smoothing and cluster structure.

Mean shift and K-means

Both methods repeatedly compute means.

K-means uses:

hard cluster membership $\rightarrow$ cluster mean.

Mean shift uses:

distance-dependent weights $\rightarrow$ local weighted mean.

K-means:

K is specified directly.

Mean shift:

K emerges from the density modes.

The bandwidth replaces K as the main scale parameter.

Computational cost

A direct mean-shift update evaluates distances from each current location to all observations. For n starting points in d dimensions over T iterations:

$$O(n^2 d T).$$

For large datasets, common approximations include:

- ▶ neighborhood search;
- ▶ compact-support kernels;
- ▶ subsampling;
- ▶ fewer starting locations.

Mean shift can therefore be substantially more expensive than K-means.

Practical modeling choices

Mean-shift results depend strongly on the geometry of the feature space.
Important choices include:

feature scaling, bandwidth, and mode-merging tolerance.

Two nearby numerical modes may be merged into one final cluster.
A mode of the estimated density is also not automatically a meaningful real-world class.

Spectral clustering

CS5487 ■ Semester A, 2026/27

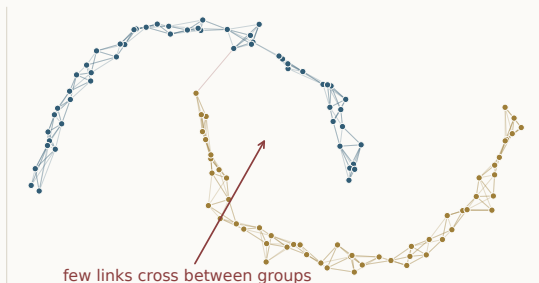
Centroid geometry has limits

K-means and GMM work well for compact groups around centers. They can struggle with curved structures, where two points may be far apart but still connected through nearby observations.

Spectral clustering uses:

local connectivity

instead of distance to one centroid.



Similarity graph

Each observation $\mathbf{x}_i$ becomes one graph node.
Nearby nodes are connected with a similarity weight:

$$w_{ij} \geq 0.$$

A common choice is:

$$w_{ij} = \exp \left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|_2^2}{2s^2} \right),$$

where $s > 0$ controls how quickly similarity decreases with distance.
Collect all weights into the similarity matrix:

$$\boxed{\mathbf{W} = [w_{ij}]}.$$

Large w_{ij} means nodes i and j are strongly connected.

Degree matrix

The similarity matrix tells us each individual connection.
For node i , its total connection strength is:

$$d_i = \sum_j w_{ij}.$$

Collect these values into the degree matrix:

$$\boxed{\mathbf{D} = \text{diag}(d_1, \dots, d_n)}.$$

Thus $\mathbf{W}$ stores pairwise similarities, while $\mathbf{D}$ stores how strongly each node is connected overall.

Local disagreement

Suppose each node receives a scalar value f_i .

For node i , compare its value with its neighbors:

$$\sum_j w_{ij}(f_i - f_j).$$

Expanding gives:

$$\sum_j w_{ij}(f_i - f_j) = d_i f_i - \sum_j w_{ij} f_j.$$

In matrix form, this is:

$$(\mathbf{D} - \mathbf{W})\mathbf{f}.$$

This motivates the graph Laplacian:

$$\boxed{\mathbf{L} = \mathbf{D} - \mathbf{W}}.$$

Meaning of the graph Laplacian

For node i :

$$(\mathbf{L}\mathbf{f})_i = \sum_j w_{ij}(f_i - f_j).$$

If node i has a value similar to its strongly connected neighbors, this quantity is small.

If strongly connected nodes receive very different values, it becomes large.

Therefore $\mathbf{L}$ measures:

variation relative to local neighbors.

This is useful because points inside the same cluster should vary little over strong graph connections.

Laplacian energy

The total graph variation is:

$$\mathbf{f}^\top \mathbf{L} \mathbf{f} = \frac{1}{2} \sum_{i,j} w_{ij} (f_i - f_j)^2.$$

Strongly connected nodes have large w_{ij} , so assigning them very different values is heavily penalized.

A small value of $\mathbf{f}^\top \mathbf{L} \mathbf{f}$ therefore means:

strongly connected nodes receive similar values.

This is the key link between the Laplacian and clustering.

Graph cuts

Suppose the graph is divided into two groups A and B .
The total weight crossing the partition is:

$$\text{cut}(A, B) = \sum_{i \in A, j \in B} w_{ij}.$$

A useful clustering should have strong connections within groups and weak connections between groups.

Therefore we seek a partition with a small cut value.

The Laplacian gives a matrix representation of this separation objective.

Laplacian energy represents a cut

Encode a two-group partition by:

$$f_i = \begin{cases} 1, & i \in A, \\ 0, & i \in B. \end{cases}$$

For nodes in the same group, $(f_i - f_j)^2 = 0$.

For nodes in different groups, $(f_i - f_j)^2 = 1$.

Therefore the Laplacian energy counts the weights crossing the partition:

$$\mathbf{f}^\top \mathbf{L} \mathbf{f} = \text{cut}(A, B).$$

So graph clustering can be expressed through the Laplacian.

Balanced graph cuts

Minimizing only $\text{cut}(A, B)$ can isolate a single weakly connected node.

Define the graph volume:

$$\text{vol}(A) = \sum_{i \in A} d_i.$$

Normalized cut balances separation and group size:

$$\text{Ncut}(A, B) = \frac{\text{cut}(A, B)}{\text{vol}(A)} + \frac{\text{cut}(A, B)}{\text{vol}(B)}.$$

This discourages trivial tiny clusters.

From graph cuts to eigenvectors

Searching over all discrete graph partitions is difficult.

Relax the discrete cluster indicator to a real-valued vector $\mathbf{f} \in \mathbb{R}^n$.

For normalized cut, the relaxed problem becomes:

$$\min_{\mathbf{f}^\top D \mathbf{1} = 1} \frac{\mathbf{f}^\top L \mathbf{f}}{\mathbf{f}^\top D \mathbf{f}}.$$

Its stationary solutions satisfy:

$$\boxed{L\mathbf{f} = \lambda D\mathbf{f}}.$$

The clustering problem is therefore converted into an eigenvector problem.

The first useful eigenvector

Because every row of L sums to zero:

$$L\mathbf{1} = \mathbf{0}.$$

The smallest eigenvalue is therefore $\lambda_1 = 0$, with a constant eigenvector.

A constant value cannot separate nodes.

The eigenvector associated with the second-smallest eigenvalue is the first non-trivial solution.

It changes slowly across strong connections and more strongly across weak connections.

For two clusters, this eigenvector reveals the main graph separation.

Spectral embedding

For multiple clusters, use several eigenvectors associated with the smallest eigenvalues.
Place them into a matrix:

$$U = [f_1, \dots, f_K].$$

Row i of U becomes a new representation of observation i :

$$\mathbf{x}_i \longrightarrow \mathbf{u}_i.$$

The new coordinates describe graph connectivity rather than the original Euclidean geometry.
Curved groups can become easier to separate in this spectral space.

Spectral clustering algorithm

A common procedure is:

1. Build a local similarity graph.
2. Form the similarity matrix W .
3. Compute the degree matrix D .
4. Form the graph Laplacian $L = D - W$.
5. Compute the low-eigenvalue eigenvectors.
6. Use their rows as spectral features.
7. Run K-means on these features.

The main transformation is:

original data $\rightarrow$ local graph $\rightarrow$ spectral representation $\rightarrow$ clusters.

Graph construction matters

- ▶ Spectral clustering depends strongly on how the similarity graph is built.
- ▶ Important choices include:
 - the number of neighbors;
 - the similarity scale s ;
 - the number of clusters K .
- ▶ If the graph is too sparse, one true group may become disconnected.
- ▶ If the graph is too dense, different groups may become strongly linked.
- ▶ Therefore, graph construction is part of the clustering model, not just a preprocessing step.

Computational cost

A fully connected similarity matrix requires roughly n^2 pairwise similarities. The eigenvector computation also adds substantial cost.

For large datasets, common approaches include:

- ▶ sparse nearest-neighbor graphs;
- ▶ approximate neighbor search;
- ▶ sparse eigensolvers;
- ▶ subsampling or landmark methods.

Spectral clustering is usually more expensive than K-means, but can represent much richer cluster geometry.

Comparison and summary

CS5487 ■ Semester A, 2026/27

Clustering models at a glance

Method	Cluster definition	Main choice	Typical limitation
K-means	points near the same centroid	K , feature scaling	non-compact geometry
GMM	points explained by the same Gaussian component	K , covariance model	distribution assumption
Mean shift	points converging to the same density mode	bandwidth	bandwidth sensitivity
Spectral	points strongly connected in a similarity graph	graph construction, K	computational cost

The same dataset can produce different clusters because each method defines cluster structure differently.

From global models to local structure

The methods form a progression in how structure is represented:

centroids $\rightarrow$ probability distributions $\rightarrow$ density modes $\rightarrow$ graph connectivity.

This week focused on the last two views:

- ▶ KDE estimates local density from nearby observations.
- ▶ Mean shift follows that density toward local modes.
- ▶ Spectral clustering uses local graph connections to reveal non-convex groups.

Week 5 summary

The main lessons are:

- ▶ Local methods avoid forcing all data into one fixed global shape.
- ▶ Bandwidth determines the resolution of KDE and mean-shift clustering.
- ▶ Graph construction determines the geometry used by spectral clustering.
- ▶ Clustering results depend on how similarity and structure are represented.

Different cluster assumptions $\Rightarrow$ different clustering results
--

Local neighborhoods reveal density and connectivity

Questions and discussion